

STAT / ML / CODE

Baseline Methods for Rating Prediction

STAT3009 · Recommender Systems

Ben Dai

CUHK · Department of Statistics and Data Science



CUHK · STAT3009 · Recommender Systems

The Netflix rating-prediction pipeline

TRAIN.CSV: OBSERVED TRIPLE

(1960, 670, 4)

User 1960 gave movie 670 four stars. The model may learn from this row.

TEST.CSV: PREDICTION PAIR

(1574, 2956, ?)

The model sees the user and movie. The rating stays hidden until evaluation.

01

Inspect

`train` has 51,161 observed triples with user, movie and rating.

then

02

Standardize IDs

Fit shared ID maps. Create `X_train`, `y_train` and `X_test`.

then

03

Fit a predictor

Learn an abstract scoring rule `f_theta` from the training arrays.

then

04

Predict and evaluate

Compute `y_pred`. Reveal `y_test` only when calculating RMSE.

then

05

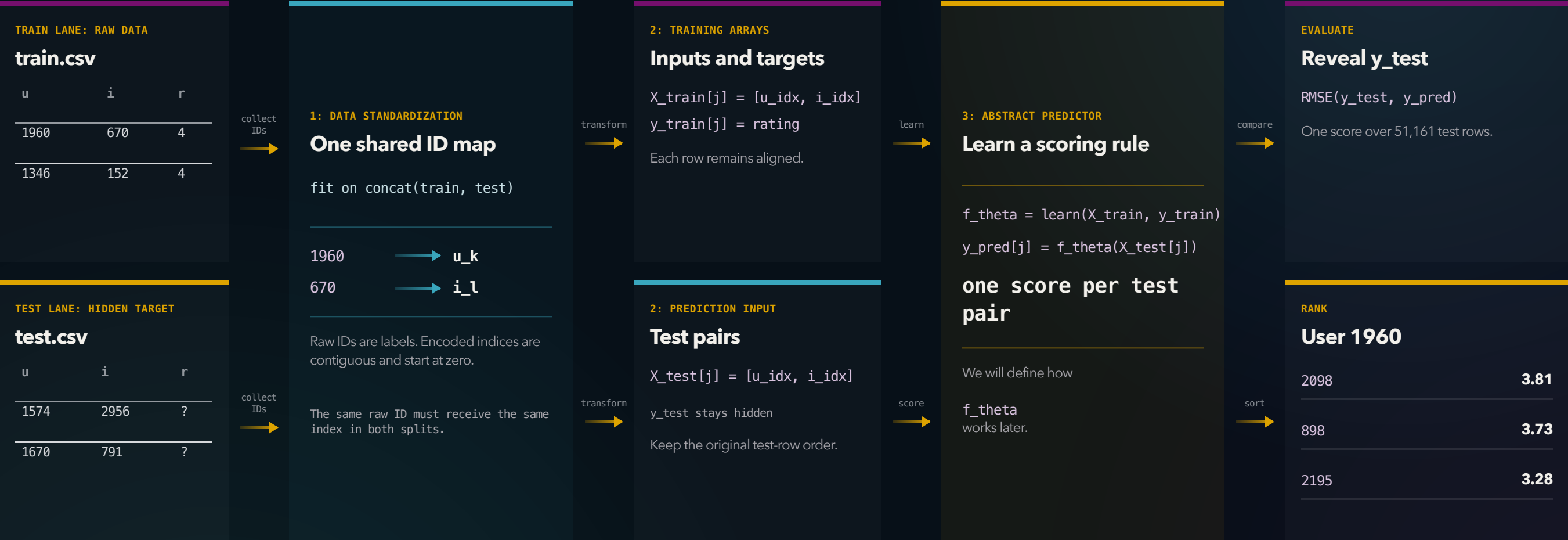
Rank

For user 1960, movies 2098, 898 and 2195 are ordered by `y_pred`.

Row alignment stays fixed: `X_test[j]` receives `y_pred[j]`, which is later compared with `y_test[j]`.



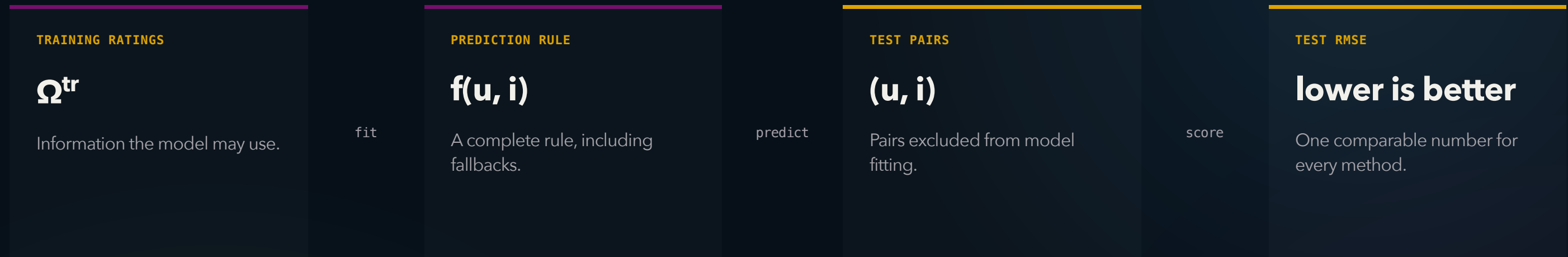
Netflix data preparation and prediction



The same ID maps transform both splits. The model learns from training ratings, then returns one prediction for every test pair.



A baseline must score every test pair



Simple

We can explain every prediction.

Complete

New users and movies still receive a score.

Reproducible

The same training data gives the same benchmark.



Data preprocessing: categorical IDs

User and item IDs are labels. The model needs stable array indices, but the spelling or numerical size of an ID carries no preference information.

USER LABELS

["U_A", "U_Z", "U_A"]

fit user vocabulary

USER INDICES

[0, 1, 0]

MOVIE LABELS

["M_B", "M_A", "M_C"]

fit movie vocabulary

MOVIE INDICES

[1, 0, 2]

RAW LABEL

"U_Z"

An identifier. Its characters carry no distance or order.



ARRAY INDEX

1

A valid zero-based position in a NumPy array.

Use two encoders: users and movies have separate vocabularies, even when both happen to be integers.



sklearn.preprocessing.LabelEncoder

PACKAGE

sklearn

MODULE

preprocessing

CLASS

LabelEncoder

from sklearn.preprocessing import LabelEncoder

```

enc = LabelEncoder()
labels = ["U_A", "U_Z", "U_A"]

enc.fit(labels)
enc.classes_          # ["U_A", "U_Z"]

enc.transform(["U_Z", "U_A"]) # [1, 0]
enc.inverse_transform([0, 1]) # ["U_A", "U_Z"]

```

01 LabelEncoder()

Creates an encoder with no learned vocabulary.

02 fit(labels)

Finds the unique labels and stores them in `classes_`.

03 transform(labels)

Applies the stored mapping to known labels.

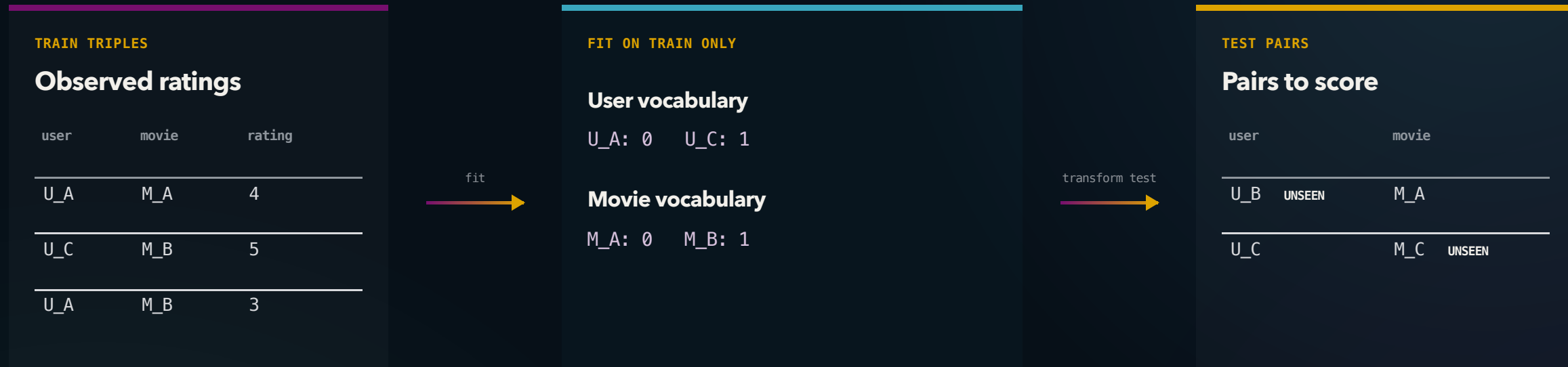
04 inverse_transform(indices)

Converts integer indices back to the original labels.

`fit` changes the encoder's state. `transform` reuses that state and cannot invent an index for an unseen label.



Train-only ID vocabulary



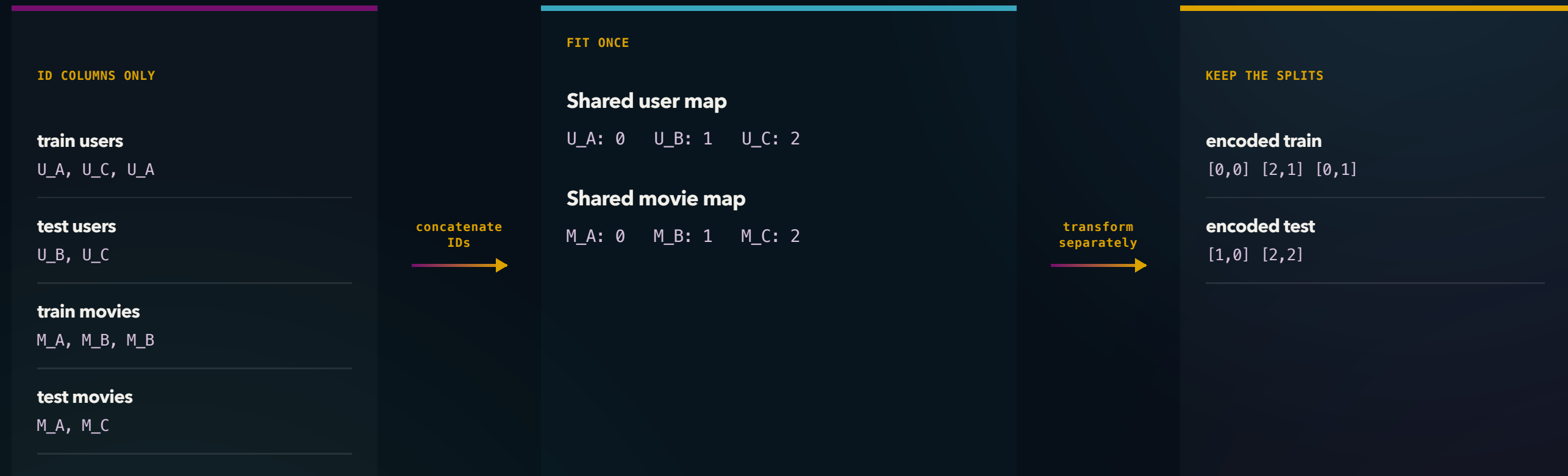
TRANSFORM CANNOT CONTINUE

U_B is absent from the user vocabulary

M_C is absent from the movie vocabulary

The course split contains test-only IDs. A train-only encoder cannot assign every scheduled test pair an index.

Shared train-test ID vocabulary



WHAT ENTERS THE ENCODER?

`user_id` and `movie_id`

WHAT STAYS OUT?

`test.rating`

Concatenation defines a common vocabulary. It does not merge the ratings or move test rows into the training set.



LabelEncoder implementation

```
from sklearn.preprocessing import LabelEncoder

all_users = pd.concat([train["user_id"], test["user_id"]])
all_items = pd.concat([train["movie_id"], test["movie_id"]])

user_enc = LabelEncoder().fit(all_users)
item_enc = LabelEncoder().fit(all_items)

train["u"] = user_enc.transform(train["user_id"])
test["u"] = user_enc.transform(test["user_id"])
train["i"] = item_enc.transform(train["movie_id"])
test["i"] = item_enc.transform(test["movie_id"])

n_users = len(user_enc.classes_)
n_items = len(item_enc.classes_)
```

01 Collect ID columns

No ratings enter all_users or all_items.

02 Fit once

Each encoder stores one shared vocabulary.

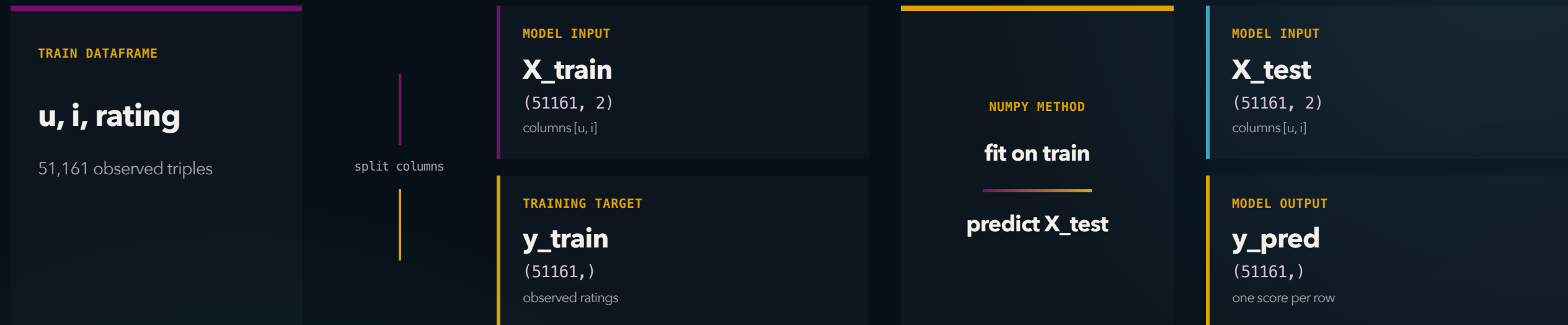
03 Transform separately

Train and test remain different datasets.

`fit` defines the mapping. `transform` applies the same mapping wherever that entity appears.



Data preprocessing: NumPy arrays



```
X_train = train[["u", "i"]].to_numpy(dtype=np.int64)
y_train = train["rating"].to_numpy(dtype=float)

X_test = test[["u", "i"]].to_numpy(dtype=np.int64)
y_test = test["rating"].to_numpy(dtype=float) # evaluation only
```

IMPLEMENTATION RULE

Pandas prepares the tables.

NumPy fits every baseline and produces **y_pred.**

During prediction, the method receives **X_test**. We reveal **y_test** only when calculating RMSE.



Netflix data and evaluation

Understand the observations, the missing pairs, and the information available at prediction time.



01



The Netflix Prize made rating prediction measurable

PRIZE

US\$1 million

for reaching the competition target

TARGET

10% improvement

over the Cinematch benchmark

TASK

Predict hidden ratings

for specified customer-movie pairs

The competition reduced recommendation to a precise offline problem: estimate a customer's missing movie rating.

Source: [Netflix Prize](#) and the [Netflix dataset README](#).



The full data and the course subset

	NETFLIX PRIZE	STAT3009 SUBSET
Training ratings	100,480,507	51,161
Users	480,189	2,000
Movies	17,770	3,568
Rating scale	1-5 stars	1-5 stars
Test labels	Hidden	Visible for teaching

PYTHON · COURSE FILES

```
both = pd.concat([train, test])

len(train)
both[["user_id", "movie_id"]].nunique()
train["rating"].agg(["min", "max"])
```

51,161 rows
2,000 users · 3,568 movies
ratings from 1 to 5

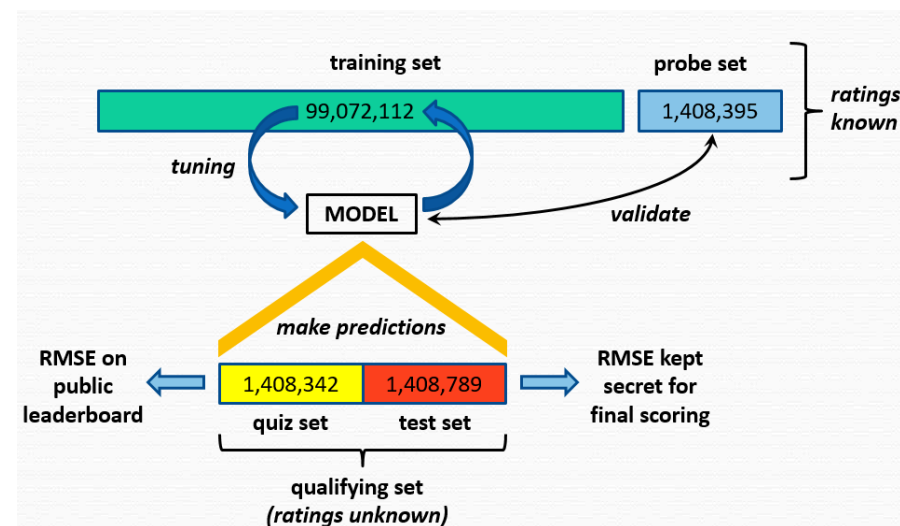
How to read the comparison

Python gives the subset counts. The original counts come from the competition README.

Sources: [Netflix dataset README](#) and [STAT3009 course data](#).



Competition split and course split



Source: Pantelis Monogioudis, Netflix Prize lecture notes.

FIT

Training · 99,072,112

Released ratings used to estimate the model.

$$100,480,507 = 99,072,112 + 1,408,395$$

VALIDATE

Probe · 1,408,395

Known ratings held out for local model checks.

The released rating total includes the fitting and probe portions.

PUBLIC FEEDBACK

Quiz · 1,408,342

Hidden ratings produced the leaderboard score.

FINAL DECISION

Test · 1,408,789

Hidden ratings determined the final ranking.

Course version

`train.csv` fits the model. We hide the `test.csv` rating column while predicting, then reveal it to calculate RMSE.



The course tables

```
import numpy as np
import pandas as pd

base = (
    "https://raw.githubusercontent.com/"
    "statmlben/STAT3009/main/"
    "dataset/netflix/"
)

fields = ["movie_id", "user_id", "rating"]
train = pd.read_csv(base + "train.csv", usecols=fields)
test = pd.read_csv(base + "test.csv", usecols=fields)
```

`train.head()`

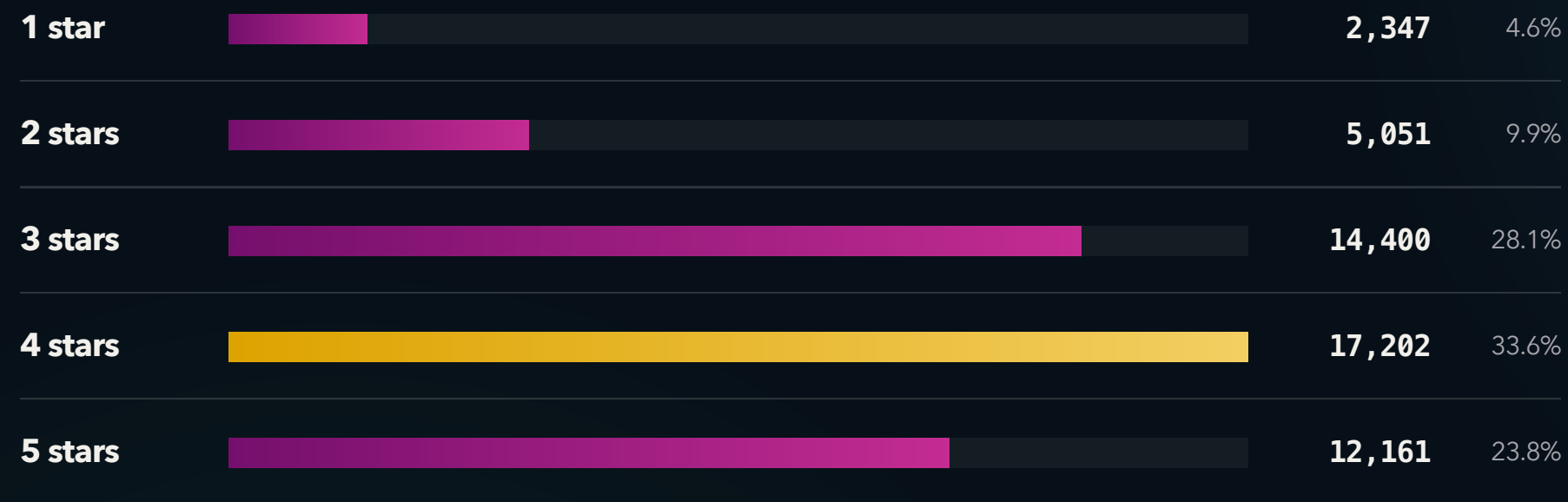
	movie_id	user_id	rating
0	670	1960	4
1	152	1346	4
2	1741	785	4
3	3032	686	5
4	536	1894	4

(1960, 670, 4) means customer 1960 gave movie 670 four stars.

IDs identify categories. Their numerical distance carries no meaning.



Rating distribution in the training set



PYTHON

```
train["rating"].value_counts(
    normalize=True
).sort_index()

train["rating"].agg(["mean", "std"])
```

TRAINING MEAN

3.621

TRAINING SD

1.088

3-5 STAR RATINGS

85.5%

The target is not centered at three stars. A credible first prediction should reflect this.



The observed matrix is 0.717% full

POSSIBLE USER-MOVIE PAIRS

2,000 × 3,568

7,136,000 cells

OBSERVED TRAINING RATINGS

51,161

one rating per observed pair

PYTHON

```
both = pd.concat([train, test])
n_user = both["user_id"].nunique()
n_movie = both["movie_id"].nunique()
```

```
density = len(train) / (n_user * n_movie)
100 * density
```

0.7169%

0.717% observed

99.283% unobserved

A blank cell means “no observed rating.” It does not mean that the customer assigned zero stars.



Interaction counts vary sharply

RATINGS PER USER IN TRAIN



RATINGS PER MOVIE IN TRAIN



PYTHON · GROUP SIZE, THEN SUMMARIZE

```
user_n = train.groupby("user_id").size()
movie_n = train.groupby("movie_id").size()
user_n.quantile([.5, .9]), user_n.max()
movie_n.quantile([.5, .9]), movie_n.max()
```

The group means summarize different amounts of evidence: some users and movies have many more ratings than others.



Train and test contain different pairs

TRAIN

51,161 pairs

Used to estimate every mean and model parameter.

0

overlapping user-movie pairs

TEST

51,161 pairs

Ratings enter the RMSE calculation only.

USERS

74 unseen in train

1,843 users occur in both files

MOVIES

584 unseen in train

2,415 movies occur in both files

PYTHON

```
keys = ["user_id", "movie_id"]
n_overlap = train.merge(test, on=keys).shape[0]
new_users = len(set(test.user_id) - set(train.user_id))
new_movies = len(set(test.movie_id) - set(train.movie_id))
```

0 overlapping pairs · **74** new users · **584** new movies



Four types of test pair

TEST PAIR	USER / MOVIE SEEN?	ROWS	SHARE
Warm pair	Yes / Yes	50,159	98.04%
New movie	Yes / No	850	1.66%
New user	No / Yes	149	0.29%
New user and movie	No / No	3	0.01%

PYTHON

```
seen_u = test.user_id.isin(train.user_id)
seen_i = test.movie_id.isin(train.movie_id)

pd.crosstab(seen_u, seen_i)
```

A held-out pair can still contain familiar IDs.

Fallbacks cover the 1,002 rows with a new user or movie.



Leakage-free evaluation

TRAIN RATINGS

Fit summaries

global mean, user effects, movie effects

use fitted values

TEST IDs

Generate predictions

one score for every test row

after prediction

TEST RATINGS

Calculate RMSE

compare truth with predictions

DATA LEAKAGE

Computing a user or movie mean from `pd.concat([train, test])` lets test ratings influence the prediction.

Use both files only to describe the dataset universe, never to fit a predictive quantity.



RMSE as the common scoreboard

$$\text{RMSE} = \sqrt{\frac{1}{n_{\text{test}}} \sum_{j=1}^{n_{\text{test}}} (y_j - \hat{y}_j)^2}$$

```
1 def rmse(y_true, y_pred):
2     error = np.asarray(y_true) - np.asarray(y_pred)
3     return np.sqrt(np.mean(error ** 2))
4
5 # test ratings enter only here
6 score = rmse(y_test, y_pred)
```

Interpretation

Typical prediction error, expressed on the 1-5 rating scale, with larger misses penalized more heavily.

EVERY METHOD RETURNS

51,161 predictions

in the same row order as `test`



From training triples to a prediction vector

TRAIN · THREE-COLUMN LIST OF (u, i, r)

u	i	r
1960	670	4
1346	152	4
785	1741	4
686	3032	5

Learn which systematic patterns explain r.

summarize

PREDICTION RULE

$$\hat{r}_{ui} = \text{rule}(u, i)$$

Use quantities calculated only from `train`.

global level

user pattern

movie pattern

apply row by row

TEST PAIRS AND ALIGNED OUTPUT

`test[["user_id", "movie_id"]]`

00 1574 2956

01 1670 791

02 837 1547

03 1943 1846

y_pred

3.84

3.35

3.43

3.44

Row j in the vector predicts row j in `test`.

`len(y_pred) == len(test)`

The model never reads `test["rating"]`. We use that column only after the vector is complete.



Mean baselines

Use progressively more specific averages while retaining a prediction for every test row.



02



Global mean baseline

ONE NUMBER FROM TRAIN

$$\mu = \frac{1}{|\Omega^{\text{tr}}|} \sum_{(u,i) \in \Omega^{\text{tr}}} r_{ui} \quad \hat{r}_{ui} = \mu$$

TRAINING MEAN

3.621

TEST RMSE

1.085

UNSEEN IDS

Always defined

The rating distribution gives us a stronger constant prediction than an arbitrary midpoint such as three stars.



Global mean from the training triples

TRAIN · 51,161 TRIPLES

u	i	r
1960	670	4
1346	152	4
785	1741	4
686	3032	5
1894	536	4
⋮	⋮	⋮

aggregate r

SUM OF ALL RATINGS

185,262

`y_train.sum()`

NUMBER OF RATINGS

51,161

`y_train.size`

GLOBAL MEAN

$$\frac{185,262}{51,161} = 3.621156...$$

CALCULATE

`global_mean = y_train.mean()`

FILL THE TEST VECTOR

`y_pred_global = np.full(X_test.shape[0], global_mean)`

The global mean uses the `r` column. It does not group by `u` or `i`.



Systematic variation across users and movies

USER RATING LEVELS

User 589	74 ratings	2.01
User 1190	74 ratings	4.99

MOVIE RATING LEVELS

Movie 2341	64 ratings	2.75
Movie 205	81 ratings	4.49

One global number leaves predictable structure in the prediction errors: some users rate low, and some movies receive stronger ratings.



User mean baseline

ONE NUMBER PER OBSERVED USER

$$\bar{r}_u = \frac{1}{|\mathcal{I}_u^{\text{tr}}|} \sum_{i \in \mathcal{I}_u^{\text{tr}}} r_{ui}$$

KNOWN USER

$$\hat{r}_{ui} = \bar{r}_u$$

same prediction for every movie

NEW USER

$$\hat{r}_{ui} = \mu$$

fall back to the global mean

TEST RMSE

1.017

CHANGE FROM GLOBAL

-0.069



User means from training triples

SELECTED TRAIN TRIPLES · USERS INTERLEAVED

u	i	r
1960	670	4
589	3181	1
1346	152	4
1960	2597	4
589	422	2
1346	2102	5

Rows with the same `u` belong to one group. The movie IDs may differ.

group by `u`

ONE AVERAGE PER USER GROUP

User	$\sum r / n_u$	User mean
1960	1,324 / 347	3.816
589	149 / 74	2.014
1346	398 / 107	3.720

Same calculation, separate groups

Each observed user receives one value. User 1960 gets $1324 / 347 = 3.815562$.

STORE ONE VALUE PER USER

```
user_mean[1960] = 3.816
user_mean[589] = 2.014
user_mean[1346] = 3.720
```



User mean in NumPy

```
user_mean = np.full(n_users, global_mean)

for u in np.unique(X_train[:, 0]):
    user_mean[u] = np.mean(y_train[X_train[:, 0] == u])

y_pred_user = user_mean[X_test[:, 0]]
```

EXAMPLE · USER $u = 1$

User column $[0, 1, 0, 2, 1]$

Boolean mask $[F, T, F, F, T]$

Selected ratings $[1, 2]$

`user_mean[1]` 1.5

Build the lookup vector from train

`X_train[:, 0] == u` selects the ratings from user u .

Predict by array indexing

Each encoded test user retrieves one value from `user_mean`.

Item mean baseline

ONE NUMBER PER OBSERVED MOVIE

$$\bar{r}_i = \frac{1}{|\mathcal{U}_i^{\text{tr}}|} \sum_{u \in \mathcal{U}_i^{\text{tr}}} r_{ui}$$

KNOWN MOVIE

$$\hat{r}_{ui} = \bar{r}_i$$

same prediction for every user

NEW MOVIE

$$\hat{r}_{ui} = \mu$$

fall back to the global mean

TEST RMSE

1.052

CHANGE FROM GLOBAL

-0.034



Item mean uses the same pattern

SELECTED TRAIN TRIPLES · MOVIES INTERLEAVED

u	i	r
1960	670	4
1346	152	4
785	1741	4
1882	670	4
657	152	2
961	1741	4

Rows with the same **i** belong to one group. The user IDs may differ.

group by i

ONE AVERAGE PER MOVIE GROUP

Movie	$\sum r / n_i$	Item mean
670	145 / 47	3.085
152	292 / 74	3.946
1741	416 / 108	3.852

Only the grouping column changes

User mean groups by column 0. Item mean groups by column 1.

ALL MOVIES · NUMPY

```
item_mean = np.full(n_items, global_mean)
for i in np.unique(X_train[:, 1]):
    item_mean[i] = np.mean(y_train[X_train[:, 1] == i])
y_pred_item = item_mean[X_test[:, 1]]
```



Three mean baselines on the test set

METHOD	WHAT VARIES?	FALLBACK	TEST RMSE
Global mean	Nothing	Always defined	1.085
User mean	User	Global mean for new user	1.017
Item mean	Movie	Global mean for new movie	1.052

User effects explain more error in this split

The user mean improves more than the movie mean.

Neither model combines both sources

User mean ignores which movie is scored. Movie mean ignores which user is scored.



Fallback behavior for unseen IDs

TEST CASE	GLOBAL	USER MEAN	ITEM MEAN
Known user, known movie	μ	$\bar{r}_u$	$\bar{r}_i$
Known user, new movie	μ	$\bar{r}_u$	μ
New user, known movie	μ	μ	$\bar{r}_i$
New user, new movie	μ	μ	μ

The global mean is more than a weak model. It is the fallback that makes the other baselines complete.

Three-mean baseline

Take the arithmetic average of the user mean, item mean, and global mean.



03



Arithmetic average of three means

THREE NUMBERS · EQUAL WEIGHT

user mean

+

item mean

+

global mean

$\bar{r}_u$

User mean

User u's average training rating.

$\bar{r}_i$

Item mean

Movie i's average training rating.

μ

Global mean

The average over every training rating.

$$\hat{r}_{ui} = \frac{\bar{r}_u + \bar{r}_i + \mu}{3}$$



Flexible weights

$$\hat{r}_{ui} = w_u \bar{r}_u + w_i \bar{r}_i + w_0 \mu, \quad w_u + w_i + w_0 = 1$$

USER WEIGHT

w_u

Controls how much the user's rating tendency contributes.

ITEM WEIGHT

w_i

Controls how much the movie's average rating contributes.

GLOBAL WEIGHT

w_0

Pulls the prediction toward the overall rating level.

CHOICE USED TODAY

$$w_u = w_i = w_0 = 1/3$$

Equal weights give a simple average. Other weights give different averages without changing the prediction workflow.



A three-number prediction



Missing user or item mean

Put the global mean into the missing slot, then average the same three positions.

Three-mean baseline in Python

```
u_test = X_test[:, 0]
i_test = X_test[:, 1]

# The mean arrays already contain the global fallback
user_term = user_mean[u_test]
item_term = item_mean[i_test]

y_pred_three = (user_term + item_term + global_mean) / 3

round(rmse(y_test, y_pred_three), 3)
```

OUTPUT

1.005

Equal weights

each mean contributes one third

Missing group mean

replace it with the global mean

Complete output

every test row receives a score



Test-set results

METHOD	TEST RMSE	REDUCTION FROM GLOBAL	INFORMATION USED
Global mean	1.085	reference	overall rating level
Item mean	1.052	0.034	movie history
User mean	1.017	0.069	user history
Three-mean average	1.005	0.081	user, movie, and global means

The equal-weight three-mean baseline reduces test RMSE by about 7.4% relative to the global baseline on this course split.



Performance by test-pair type

TEST-PAIR TYPE	ROWS	THREE-MEAN RMSE	PREDICTION USED
Known user, known movie	50,159	1.000	$(\bar{r}_u + \bar{r}_i + \mu) / 3$
Known user, new movie	850	1.264	$(\bar{r}_u + 2\mu) / 3$
New user, known movie	149	1.114	$(\bar{r}_i + 2\mu) / 3$
New user, new movie	3	0.621	μ only

The final row has only three observations.

Its small RMSE is sampling noise, not evidence that the global mean handles full cold start well.



Prediction scores and ranking

Return from rating prediction to the ordered recommendation list.



04



From prediction scores to a ranking

EXAMPLE TEST USER	User 1960	347 training ratings · training mean 3.82			
CANDIDATE MOVIE	GLOBAL	USER MEAN	ITEM MEAN	THREE-MEAN	OBSERVED TEST RATING
Movie 2098	3.62	3.82	4.00	3.81	4
Movie 898	3.62	3.82	3.75	3.73	4
Movie 2195	3.62	3.82	2.40	3.28	2

Global and user means produce ties

They cannot order movies for one user.

Three-mean scores vary by movie

Sorting the three-mean column gives 2098, 898, 2195.

For one user, the user and global terms are constant. The three-mean score therefore preserves the item-mean order while pulling scores toward the center.

Source: predictions calculated from the STAT3009 course files. Test ratings appear only for retrospective evaluation.



What the three-mean model cannot explain

THREE-MEAN MODEL

$$(\mu + \bar{r}_u + \bar{r}_i) / 3$$

Separate average tendencies with equal weights.

leaves

UNEXPLAINED ERROR

$$r_{ui} - \hat{r}_{ui}$$

Preference specific to this user-movie pairing.

A strict rater can still love one movie.

The user's low average pulls every prediction down, but it cannot represent a special match.

A popular movie can still be wrong for one user.

The movie's high average pulls every prediction up, but it cannot represent individual incompatibility.

Collaborative filtering models the user-movie interaction that separate averages cannot represent.



Baselines turn data properties into modeling choices

01 Rating imbalance sets the global level

02 User and movie variation motivates group effects

03 Cold-start rows require explicit fallbacks

04 Unexplained user-movie interaction motivates collaborative filtering

bendai.org/CUHK-STAT3009 ↗

